AI-Driven Portfolio Protection

Simple-Modelling Track — Chapters 4 and 5 (working draft)

Fiyinfoluwa Akano
URN 6962514
Supervisor: Dr Cuong Nguyen

August 4, 2026

Chapter 4

System Architecture & Implementation

This chapter describes how the mathematical model of Chapter 3 becomes running Python code. The implementation translates the Markov Decision Process (MDP) into a modular execution system built on Gymnasium and PyTorch, separating market state handling, policy evaluation, transaction verification, and performance evaluation. Everything here is verifiable on the public orphan branch `simple-modelling-clean` of `https://github.com/TheFinix13/dissertation-project`.

4.1 Overview of system design

Figure 4.1 shows the single loop everything else plugs into. The policy network $\pi_\theta(a \mid s)$ proposes a discrete action a_t; the trading environment (a Gymnasium `gym.Env` subclass) executes the action against the market data stream and returns the next state s_{t+1} and reward R_t. Training is running this cycle many times and adjusting θ so actions that increased wealth become more probable.

Figure 4.2 places the same loop in the actor–critic architecture used by A2C and PPO: a shared observation feeds a policy head (actor) and a value head (critic). The actor proposes a_t; the critic estimates $V(s_t)$ so the policy update can be written as an advantage rather than a raw Monte-Carlo return.

4.2 Environment engine (`TradingEnv`)

The core environment subclasses `gym.Env` and handles market data loading, position tracking, order checks, and state formulation. Experimental progression scales incrementally from the Iteration 1 baseline to Iteration 2.

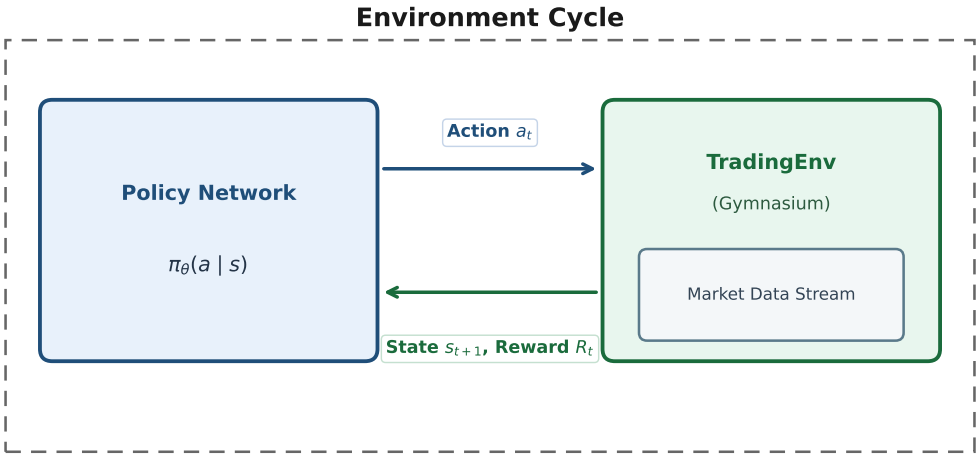

Figure 4.1: Environment cycle: the policy network exchanges actions for states and rewards with the Gymnasium trading environment wrapping the market data stream.

4.2.1 Iteration 1 baseline engine (`TradingEnv-v0`)

Implemented in `experiments/iteration1/env.py`. State:

$$s_t = [\,\Delta P_t,\ C_t,\ n_t\,]^\top. \tag{4.1}$$

Action set $\mathcal{A} = \{0 : \text{Hold},\ 1 : \text{Buy one},\ 2 : \text{Sell one}\}$ with a feasibility mask. Wealth $W_t = C_t + n_t P_t$; reward $r_t = W_{t+1} - W_t$.

4.2.2 Iteration 2 extended engine (`TradingEnv-v1`)

The 3-D state leaves three gaps identified in the Iteration-2 methodology notes: (A) the network cannot multiply $n_t \times P_t$ on the fly to judge dollar exposure; (B) once holding, there is no memory of the purchase price; (C) without episode progress, a Buy at bar 1 looks identical to a Buy at bar T.

`TradingEnv-v1` (`experiments/iteration1/env_v1.py`) closes the gaps with a scaled 5-D state:

$$s_t = \left[\,\Delta P_t,\ \mathrm{PnL}_t,\ \tau_t,\ \frac{C_t}{C_0},\ \frac{n_t P_t}{C_0}\,\right]^\top, \tag{4.2}$$

where PnL_t is unrealized return vs average entry price (zero when flat) and $\tau_t = t/T \in [0,1]$. Actions, fees, masking and reward are identical to Iteration 1, so performance differences are attributable to the state alone.

4.3 State transition mechanics

With transaction cost $c = 0.0005$ (5 bps):

- **Buy** (a_t=1): valid if $C_t \geq P_t(1+c)$; then $C \leftarrow C - P(1+c)$, $n \leftarrow n+1$.
- **Sell** (a_t=2): valid if $n \geq 1$; then $C \leftarrow C + P(1-c)$, $n \leftarrow n-1$; entry-cost reset when flat.
- **Hold** (a_t=0): balances unchanged.

Infeasible requests are masked and, if issued anyway, executed as Hold. The rate of illegal requests after training is reported in Chapter 5 as a constraint-awareness diagnostic.

4.4 Algorithm implementations

Five algorithms span value-based and policy-based families so that the choice of policy-gradient methods for trading is evidenced, not asserted.

1. **Tabular Q-learning** (`run_cartpole_qlearning.py`). Bellman update on a lookup table after hand-binning CartPole (6×6×12×12). Demonstrates why tables cannot scale to continuous trading states.
2. **DQN** (Stable-Baselines3). Network Q_ϕ trained by MSE to the Bellman target with replay and a frozen target net.
3. **REINFORCE** — from scratch in PyTorch (`experiments/phase0_games/reinforce.py`): $L(\theta) = -\sum_t \log \pi_\theta(a_t \mid s_t)\, G_t$. Auditable; not an API black box.
4. **A2C** (Stable-Baselines3). Actor–critic advantage replaces the Monte-Carlo return.
5. **PPO** (Stable-Baselines3). Clipped ratio $\min(\rho_t A_t,\ \text{clip}(\rho_t, 1 \pm \epsilon) A_t)$ with ϵ=0.2. Carried into trading.

Figure 4.3 places the supervised training template next to the PPO update so the viva mapping (predict → score → backward → step) is explicit.

4.5 Phase 0: game correctness sandbox

A trading agent that loses money is ambiguous: the market may be unlearnable, or the code may be wrong. Games remove the ambiguity because the correct outcome is known. Phase 0 validates every algorithm on three Gymnasium environments of increasing difficulty before any market data is used:

- **CartPole-v1**: 4-D state, 2 actions, +1/step upright (cap 500; solved ≈ 475).

- **FlappyBird-v0**: 12-feature state, 2 actions, sparse pipe rewards.
- **LunarLander-v3**: 8-D state, 4 discrete engines, shaped landing reward (solved $\approx$ 200).

Suites live under `experiments/phase0_games/`. Results are in Chapter 5.

4.6 Data pipeline and episode construction

Market data is SPY daily adjusted close, 2018–2024, fetched by `fetch_spy_daily.py` and frozen on disk. One episode = one calendar month of daily closes ($T \approx 18$–23). This is stated honestly: it is not the 390-minute intraday session of the idealised Chapter 3 episode; the environment API is horizon-agnostic, so minute bars are a drop-in upgrade. Months split chronologically — 58 train, 26 test (Nov 2022 – Dec 2024) — and are never shuffled.

4.7 Testing and reproducibility

Unit tests (`experiments/iteration1/test_env.py`) enforce on both engines: (i) accounting identity — summed rewards equal total ΔW within 10^{-6}; (ii) mask correctness — Buy/Sell illegality and forced Hold; (iii) v1 feature correctness — analytic PnL and τ on a hand-built price path. Every training run pins seed, budget, fee and split inside its results JSON; charts regenerate from those JSONs via `reports/builders/`.

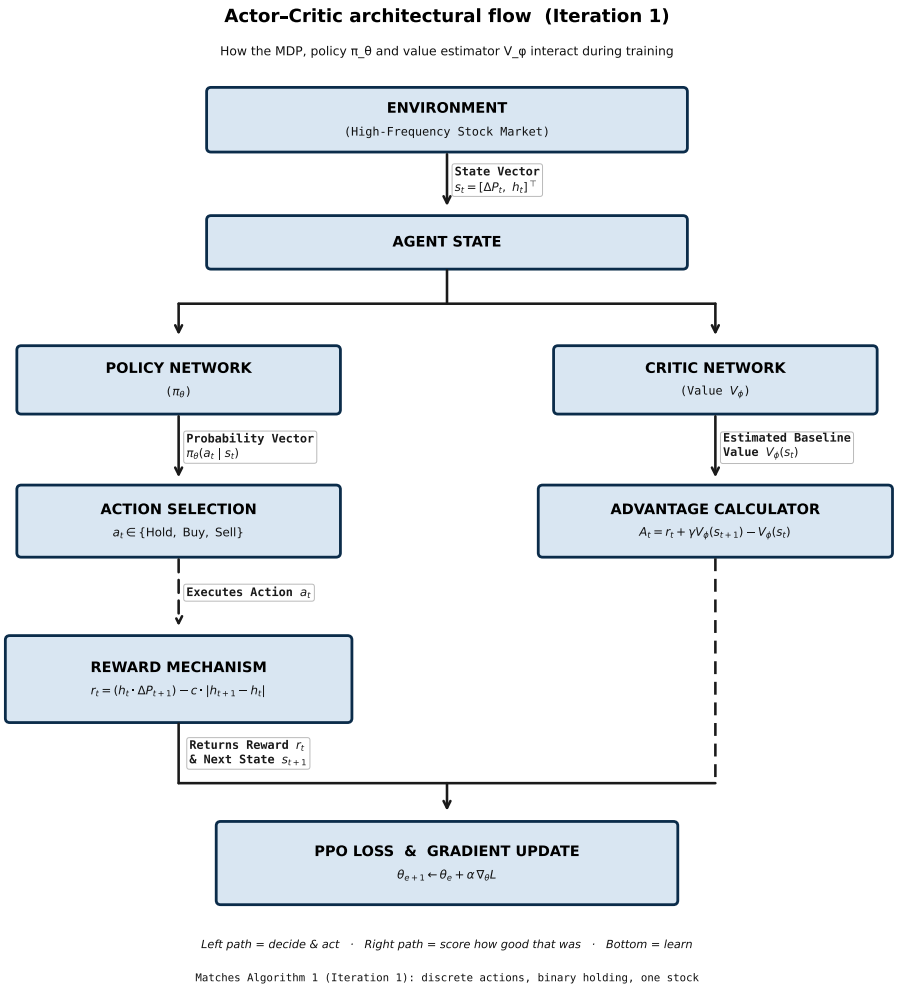

Figure 4.2: Actor–critic architecture used by A2C and PPO. The actor is the trading policy; the critic supplies the baseline for the advantage estimate.

From supervised learning to our RL training loop

Dr Nguyen's board shows classification as four beats: **predict** → **score** → **differentiate** → **update**. The same beats appear in our trading agent; only the data and the score change.

Supervised objective (his board)

$$\min_{\theta} \frac{1}{N} \sum_{n=1}^{N} \ell\big(f(x_n;\theta), y_n\big)$$

RL objective (Iteration 1 trading)

$$\max_{\theta} \frac{1}{K} \sum_{k=1}^{K} \sum_{t=0}^{T-1} \gamma^t r_t^{(k)} \qquad \text{with} \quad a_t \sim \pi_\theta(a_t \mid s_t), \quad r_t = W_{t+1} - W_t, \quad W_t = C_t + n_t P_t.$$

His board (classification)

```
net = ResNet18()
opt = Adam(...)

# data (x, y) fixed
preds = net(x)           # f(x; theta)
loss  = MSE(preds, y)/N

opt.zero_grad()
loss.backward()          # grad loss
opt.step()               # theta <- theta - a*grad
```

Ours (trading / PPO)

```
policy = PolicyNet()     # pi_theta
critic = ValueNet()      # V_phi
opt_pi, opt_V = Adam(), Adam()

# data created by acting
B = ROLLOUT(prices, pi)
A = advantages(B, V)
L_pi = - PPO_CLIP(pi, B, A)
L_V  = MSE(V(s), returns)

opt_pi.zero_grad(); L_pi.backward(); opt_pi.step()
opt_V.zero_grad();  L_V.backward();  opt_V.step()
```

Key translation

- Fixed labels y_n become rewards r_t collected online.
- One loss ℓ becomes policy objective L^{CLIP} plus value loss L^V.
- Minimise prediction error becomes maximise expected wealth change (via PPO).
- Dataset independent of θ becomes trajectories that depend on π_θ.

Use with Algorithm 3.1. Algorithm 3.1 is the full day-loop. This figure is the compact "how to get the policy" template under **Objective function** → **How to get policy**.

Figure 4.3: Supervised loss board (left) mapped onto the PPO policy / value loop (right).

Chapter 5

Empirical Results & Analysis

This chapter reports every experiment on the simple-modelling track in the order the methodology builds: Phase 0 validates the learning algorithms on games with known outcomes; Phase 1 applies the validated PPO loop to real SPY data under Iteration 1 (3-D state) and Iteration 2 (5-D state). All numbers are produced by committed scripts; none are illustrative placeholders.

5.1 Experimental setup and evaluation metrics

Phase 0 (games). Fixed seed 42. Budgets: 50,000 environment timesteps on CartPole; 150,000 on Flappy Bird and LunarLander; REINFORCE uses an equivalent episode budget. Evaluation: 30 greedy episodes. Metric: mean evaluation return.

Phase 1 (trading). SPY daily closes 2018–2024, chronological 58 / 26 month split (test = Nov 2022 – Dec 2024). $C_0 = \$10{,}000$, $c = 5\,\text{bps}$, PPO 80,000 timesteps, seed 42. Metrics: wealth change ΔW (\$), trades per month, win rate vs the strongest passive baseline. Standing diagnostics: accounting identity (tolerance 10^{-6}) and illegal-action request rate.

5.2 Phase 0: algorithm validation on games

Table 5.1 and Figures 5.1–5.4 summarise the three-game suite.

Carry-forward. Every implementation clears its random floor. Scratch REINFORCE works but is noisy — the textbook motivation for actor–critic and clipping. Tabular Q-learning reaches 500 on CartPole *only after* hand-binning; trading states with continuous cash and prices make that table explode, so Phase 1 uses policy-gradient methods

Table 5.1: Phase 0 mean evaluation return (30 greedy episodes, seed 42). CartPole solved ≈ 475; LunarLander ≈ 200. Tabular Q-learning only on discretized CartPole.

Algorithm	**CartPole-v1**	**FlappyBird-v0**	**LunarLander-v3**
Random	28.8	−7.4	−183.4
Tabular Q-learning	**500.0**	—	—
DQN	**500.0**	7.0	**177.6**
REINFORCE (scratch)	292.2	7.1	15.3
A2C	**500.0**	4.6	−41.2
PPO	**500.0**	**12.6**	176.4

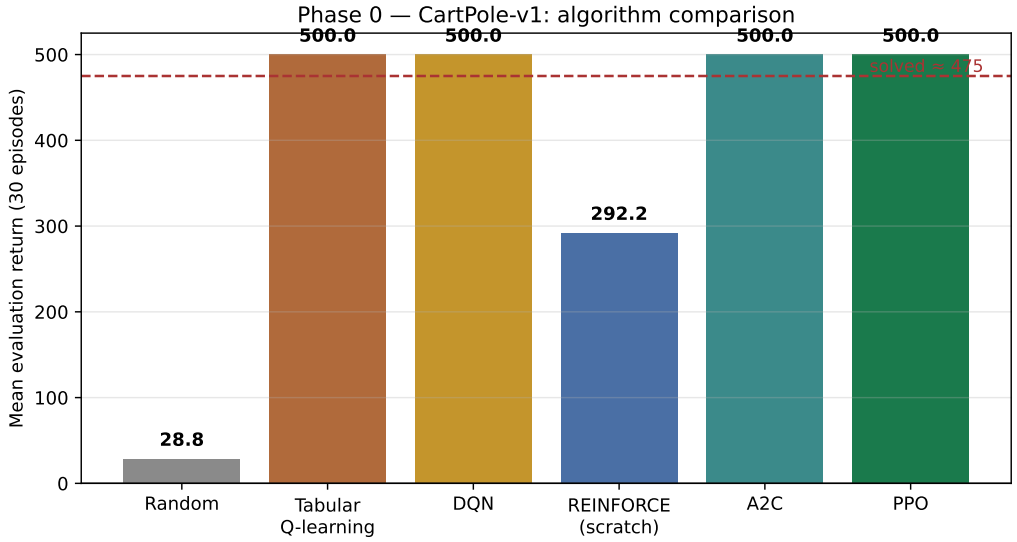

Figure 5.1: CartPole algorithm comparison. Learning methods clear the random floor; Q-table / DQN / A2C / PPO hit the 500-step cap.

with function approximation. PPO is the most consistent learner across games and is therefore the trading algorithm.

5.3 Phase 1, Iteration 1: the 3-D state agent

Table 5.2 and Figure 5.5 compare PPO on `TradingEnv-v0` against four baselines with explicit objectives.

Honest headline. Deterministic PPO matched B1b on **all 26** test months: under pure ΔW with a 3-D state, the learned policy is "get fully invested and stay there." Against the one-share baseline B1 this would look like a 10× win; against fair B1b it is rediscovered buy-and-hold. Accounting identity held exactly; illegal-action request rate was 3.9%.

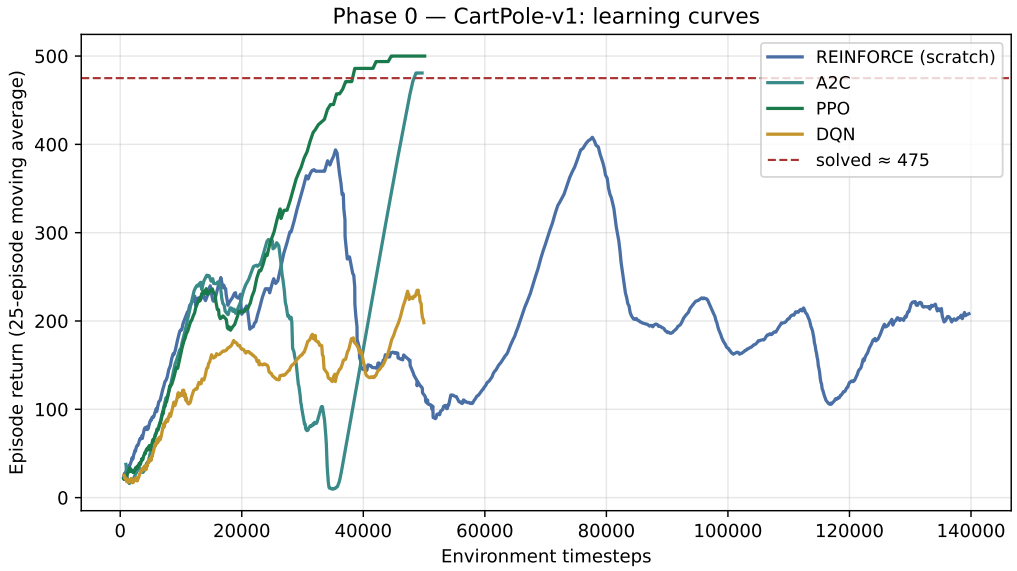

Figure 5.2: CartPole learning curves (25-episode moving average). Scratch REINFORCE learns but plateaus below the clipped / actor–critic methods.

Table 5.2: Iteration 1 on 26 held-out SPY test months. ΔW in dollars; fee 5 bps; PPO 80k steps.

Method	**Mean ΔW**	**Std**	**Trades/mo**	**Win vs B1b**
B0 do-nothing	0.00	0.00	0.0	31%
B1 buy-one-hold	7.97	16.67	1.0	31%
B1b buy-max-hold	76.91	178.68	19.1	—
B2 random (masked)	14.14	32.93	12.8	31%
A1 PPO (3-D state)	76.91	178.68	19.1	0%

5.4 Phase 1, Iteration 2: the 5-D state agent

Iteration 2 re-trains the identical PPO configuration on `TradingEnv-v1`, changing only the state. Table 5.3 and Figure 5.7 report the comparison.

Table 5.3: Iteration 2 vs Iteration 1 on the same 26 test months, budget, fee and seed.

Method	**Mean ΔW**	**Trades/mo**	**Months identical to B1b**
A1 PPO, 3-D (Iter. 1)	76.91	19.1	26 / 26
A2 PPO, 5-D (Iter. 2)	0.00	0.0	0 / 26
B1b buy-max-hold	76.91	19.1	—

The richer state *did* break the buy-and-hold collapse (0/26 months identical to B1b) but converged to the opposite corner: a cautious, mostly-flat policy. Mean action probabilities on a representative test month: Hold 0.66, Buy 0.12, Sell 0.22 — so argmax evaluation never trades. Sampling actions stochastically yields mean $\Delta W = +\$1.87$. Month-by-month, the flat agent still beats fully-invested B1b in 8 of 26 months (the

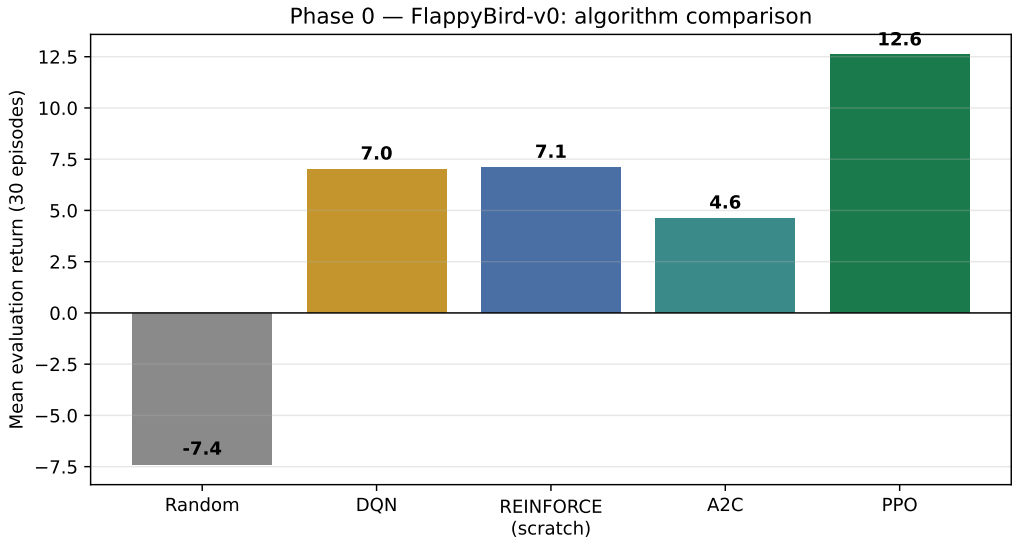

Figure 5.3: Flappy Bird comparison. PPO is the strongest learner under the fixed budget.

falling ones).

5.5 Master comparison across iterations

Table 5.4: Master results table for the simple-modelling track (Phase 1, SPY test months).

Metric	B1b buy-max	Iter. 1 PPO (3-D)	Iter. 2 PPO (5-D)
State dimension	—	3	5
Mean ΔW ($)	76.91	76.91	0.00
Mean trades / month	19.1	19.1	0.0
Identical to B1b (months)	—	26/26	0/26
Accounting identity	pass	pass	pass

5.6 Analysis and key findings

1. The reward, not the state, sets the incentive. Under $r_t = \Delta W_t$ in a mostly-rising test window, full investment is near-optimal and Iteration 1 found exactly that. Adding entry and time context moved the policy along the exposure axis (always-invested $\rightarrow$ almost-never-invested) without creating selective timing. Both extremes are rational responses to a risk-neutral reward estimated from noisy monthly episodes whose training window includes the COVID crash and the 2022 bear market.

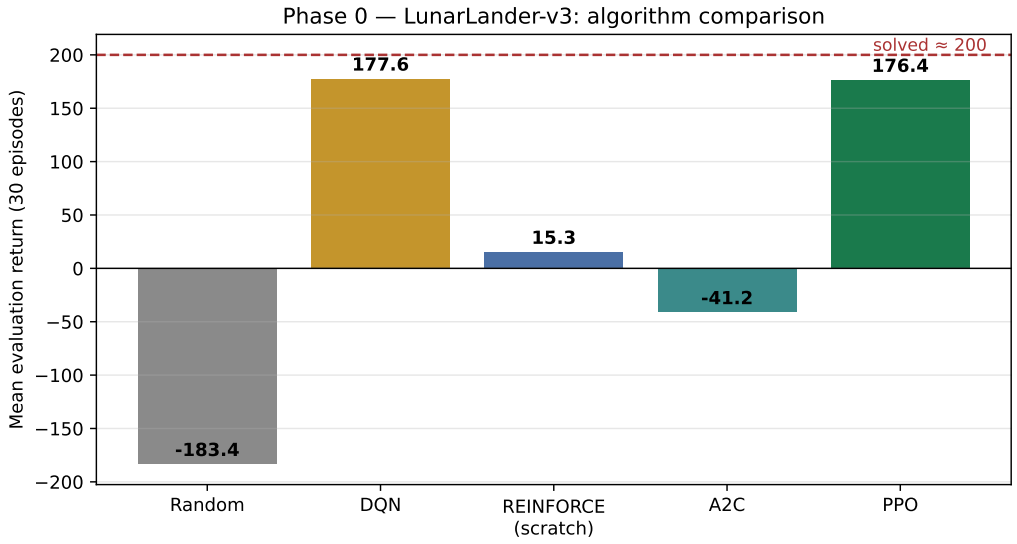

Figure 5.4: LunarLander-v3 comparison. PPO and DQN approach the solved threshold (≈ 200).

2. Baseline design is itself a result. Without fair B1b, Iteration 1 would have been reported as "PPO beats buy-and-hold by 10×." With it, the correct statement is "PPO rediscovered buy-and-hold."

3. What Iteration 3 must change. Because the state upgrade only shifted exposure, the next iteration targets the objective: $r_t = \Delta W_t - \lambda D_t$ (drawdown penalty) gives a reason to hold *selectively*. The masked-action rate and accounting identity carry forward as standing diagnostics.

Scope. Single-seed, fixed budget, monthly daily-bar episodes. Multi-seed confidence intervals, minute-bar episodes (T=390), and the drawdown-penalised reward are the planned extensions, in that order.

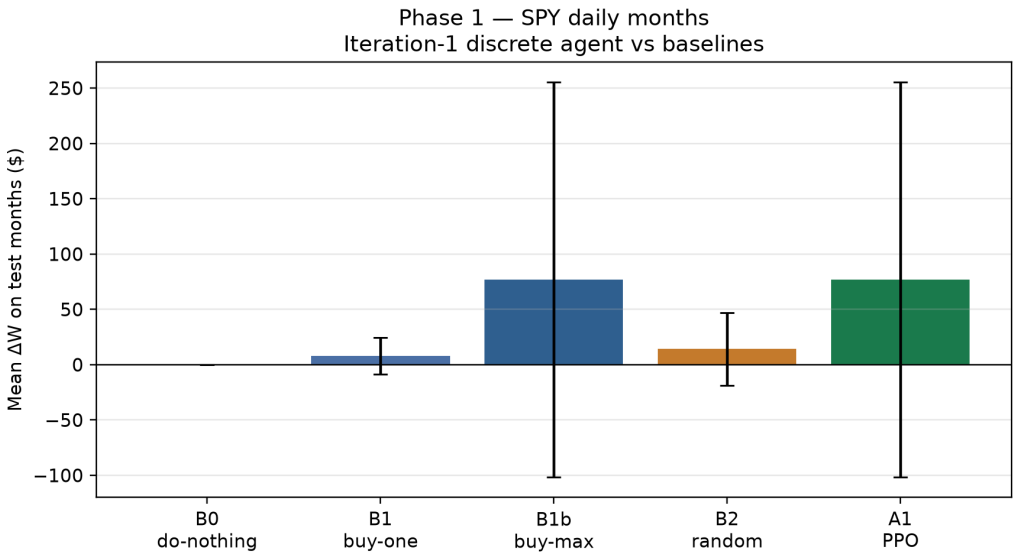

Figure 5.5: Iteration 1 mean ΔW on test months. A1 matches B1b exactly once the fair fully-invested baseline is included.

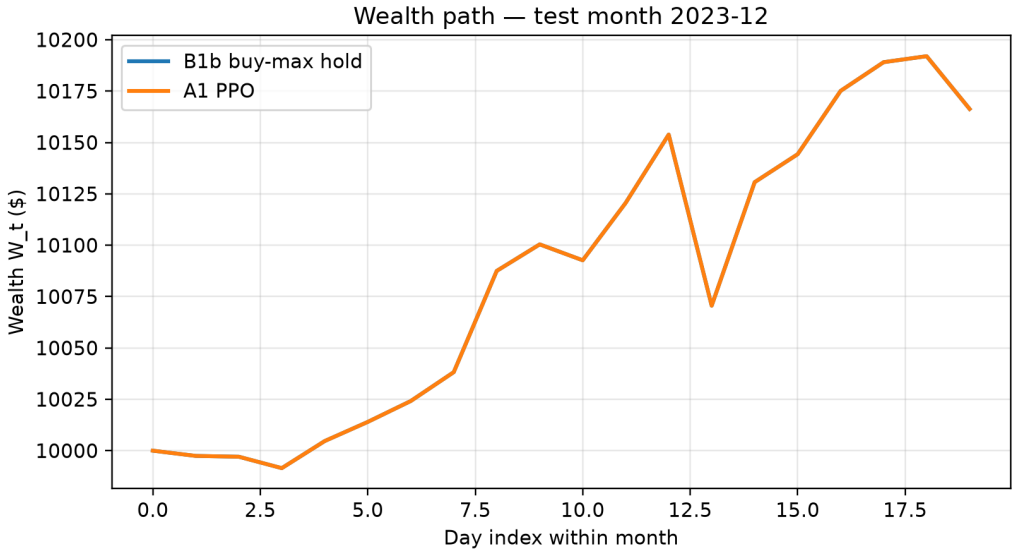

Figure 5.6: Wealth path for a representative test month: B1b buy-max hold vs A1 PPO (identical after collapse).

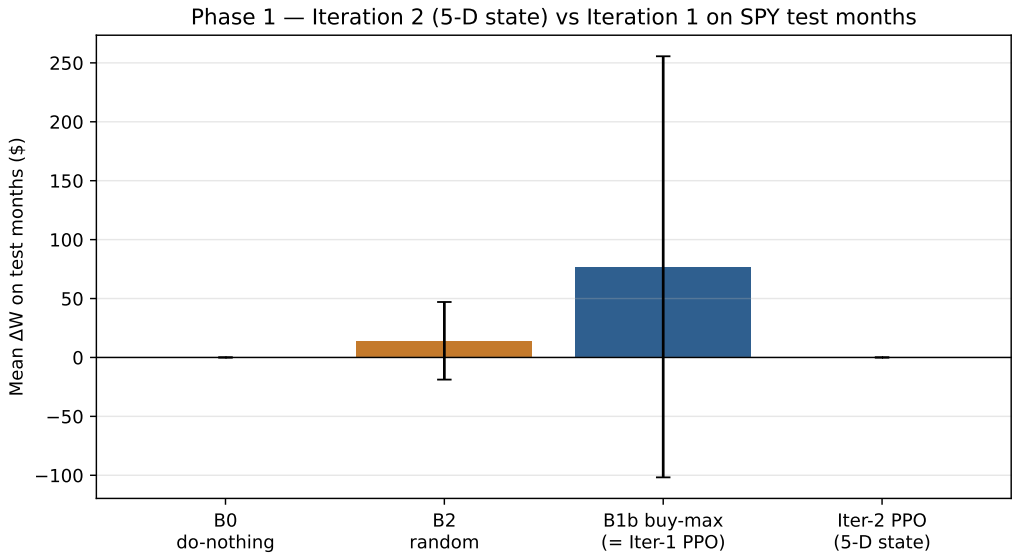

Figure 5.7: Iteration 2 (5-D) vs Iteration 1 / buy-max. Richer state breaks the collapse but converges mostly-flat under deterministic evaluation.